

Jocul Sokoban

Introducere

Am inceput proiectul cu implementarea algoritmului beam search. Pentru a optimiza algoritmul pentru jocul Sokoban, am introdus un mecanism de memorare a starilor deja vizitate pentru a evita explorarea inutila si am folosit euristici pentru a ghida cautarea catre solutii mai bune. De asemenea, am salvat traseul parcurs pentru fiecare stare pentru a putea reconstrui solutia si am numarat starile explorate pentru a analiza performanta. In plus, am folosit o structura de tip heap pentru a reduce costul de sortare si a pastra cele mai bune K stari.

In cazul algoritmului lrt_star, am adaugat o limita pentru numarul maxim de iteratii pentru a preveni buclele infinite, penalizand miscarile de pull prin cresterea costului.

Euristici utilizate:

1. misplaced_boxes_heuristic:

La inceputul proiectului, am implementat o euristica simpla, misplaced_boxes_heuristic. Aceasta euristica numara doar numarul de cutii care nu sunt pe pozitii tinta. Totusi, in timpul testelor am observat ca aceasta euristica este lenta si ineficienta. Fara vreo informatie despre cat de aproape sau departe erau cutiile de tintele lor, nu puteam face un drum bun pentru jucator. Aceasta cauza mutari aleatorii, inutile, generand arbori de cautare uriasi. Astfel, era necesara o euristica bazata pe distante, unde planificarea drumului conteaza.

- Euristica Manhattan:

Pentru a rezolva problemele primei euristici, am trecut la manhattan_heuristic. Aceasta calculeaza suma distantelor Manhattan de la fiecare cutie la cea mai apropiata tinta. Prin aceste calcule, algoritmul poate prefera stari mai aproape de scop. Au aparut imbunatatiri, unele teste care inainte oscilau, au inceput sa arate progres. Totusi, hartile au deseori pereti si blocaje pe care distanta Manhattan nu le ia in calcul. De exemplu, doua cutii

separate de un perete pot avea totusi o distanta Manhattan mica. Aceasta ducea la probleme noi: jucatorul pierdea multe mutari impingand cutii in colturi sau blocaje pentru ca euristica sugera distanta mica. Desi mai buna decat numararea simpla, aceasta euristica inca nu era constienta de harta.

- Euristica Euclidiană:

Am implementat `euclidean_heuristic`, care foloseste distanta in linie dreapta intre cutii si tinte. Desi distantele euclidiene imbunatateau usor comportamentul pe harti deschise, nu aduceau avantaj major pe hartile pline de pereti. Astfel a devenit clar ca simpla masurare a distantelor nu este suficienta; euristica trebuie sa cunoasca harta, impreuna cu peretii, respectiv deadlock urile ce pot aparea.

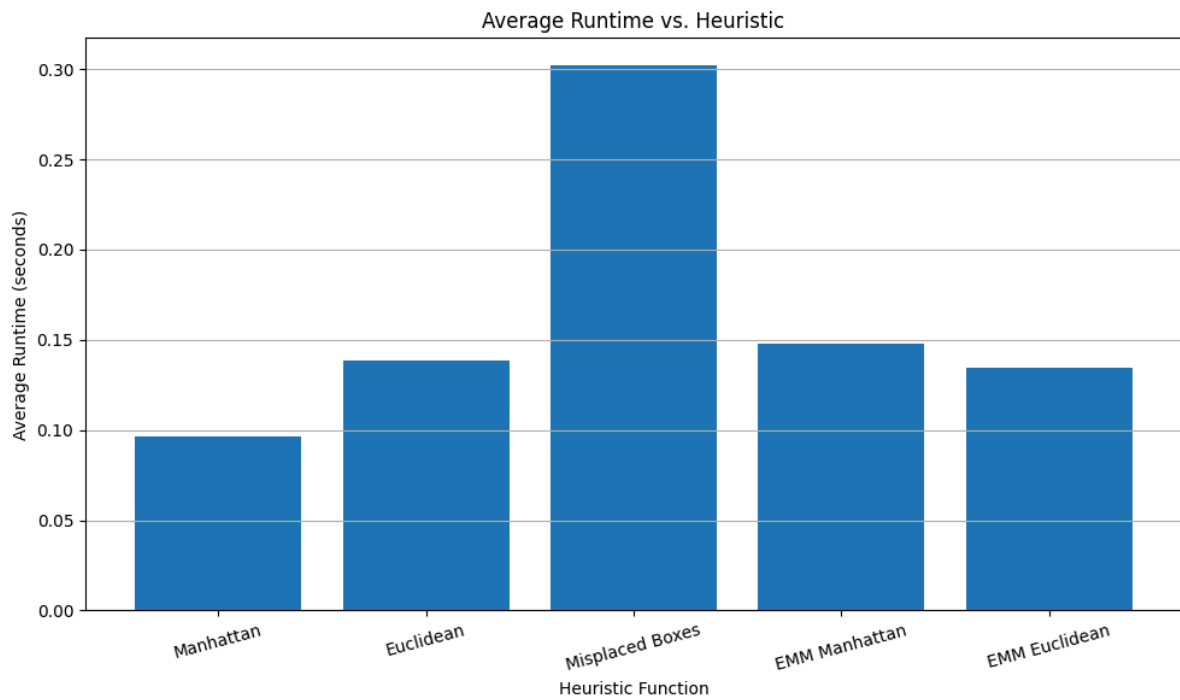
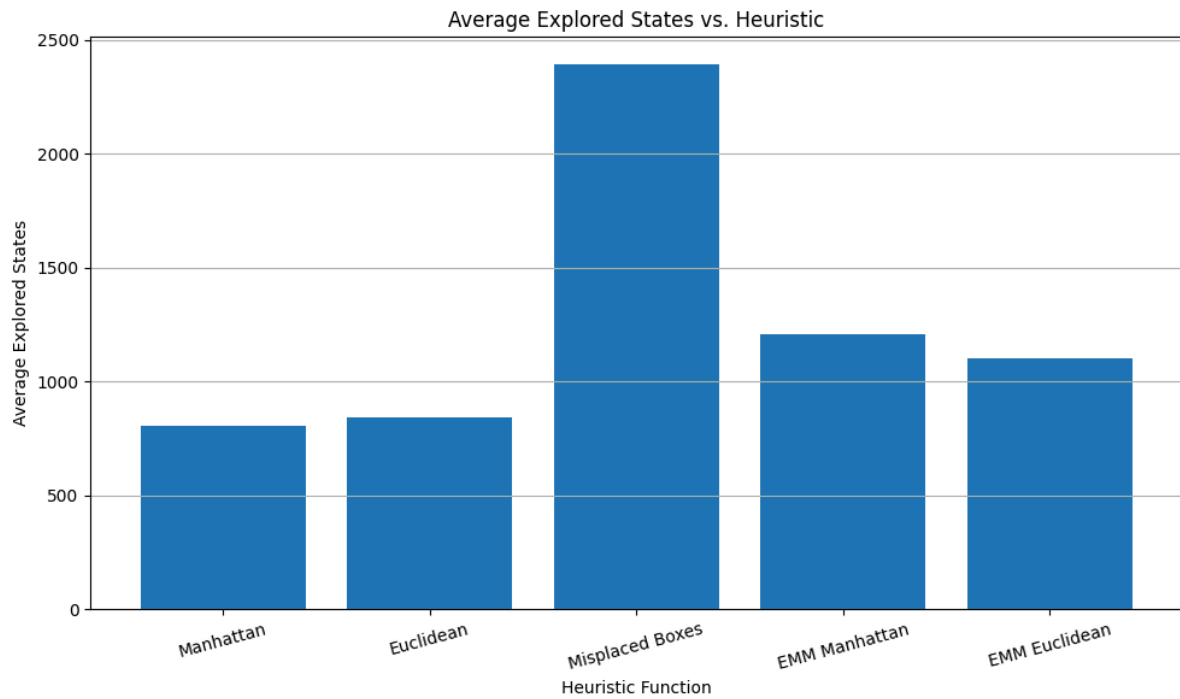
- Euristica EMM Manhattan:

Ulterior, am implementat `emm_manhattan_heuristic`. Ideea era sa asociem fiecare cutie cu cea mai apropiata tinta disponibila, evitand ca mai multe cutii sa urmareasca aceeasi tinta. Aceasta schimbare a fost buna, deoarece euristica nu evalua doar cat de aproape erau cutiile, dar le si distribuia eficient, evitand stari in care cutiile se incurcau una pe alta. In hartile cu mai multe cutii decat tinte, aceasta metoda a prevenit mutarile aleatorii. Jucatorul incepea sa rezolve portiuni ale hartii mai eficient, cutie cu cutie, in loc sa spere la progres prin noroc.

- Euristica EMM Euclidiană:

Avand in vedere ca euristica cu distanta euclidiene a adus rezultate mai bune, urmatoarea decizie logica a fost sa utilizez `emm` cu distante euclidiene.

Beam_Search:



Success Rates:

Manhattan: 77.78%

Euclidean: 77.78%

Misplaced Boxes: 66.67%

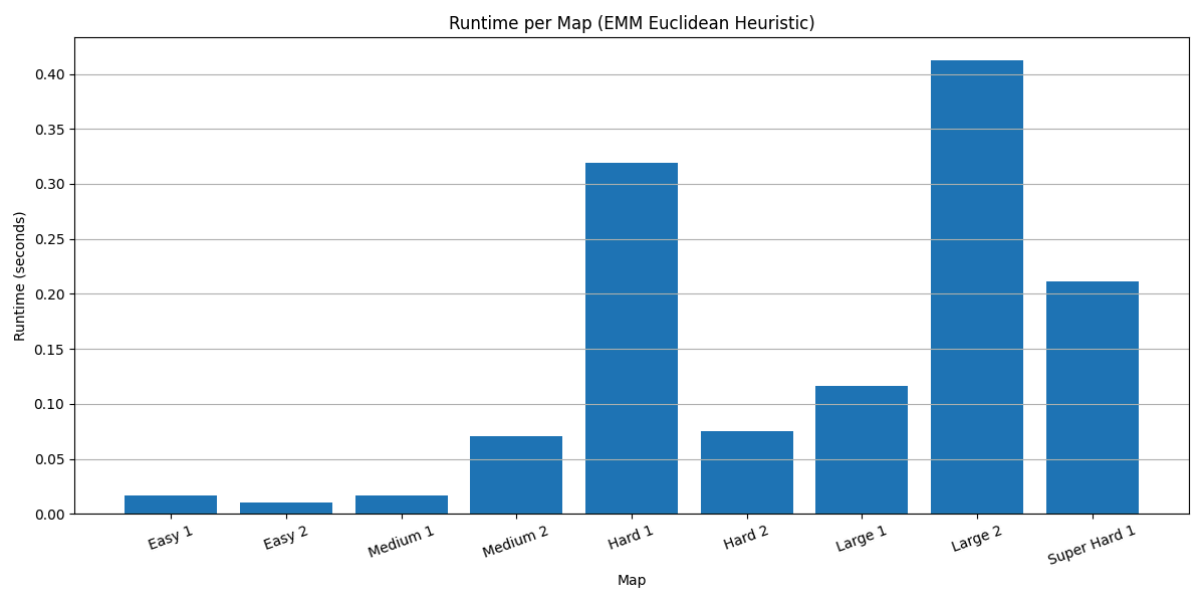
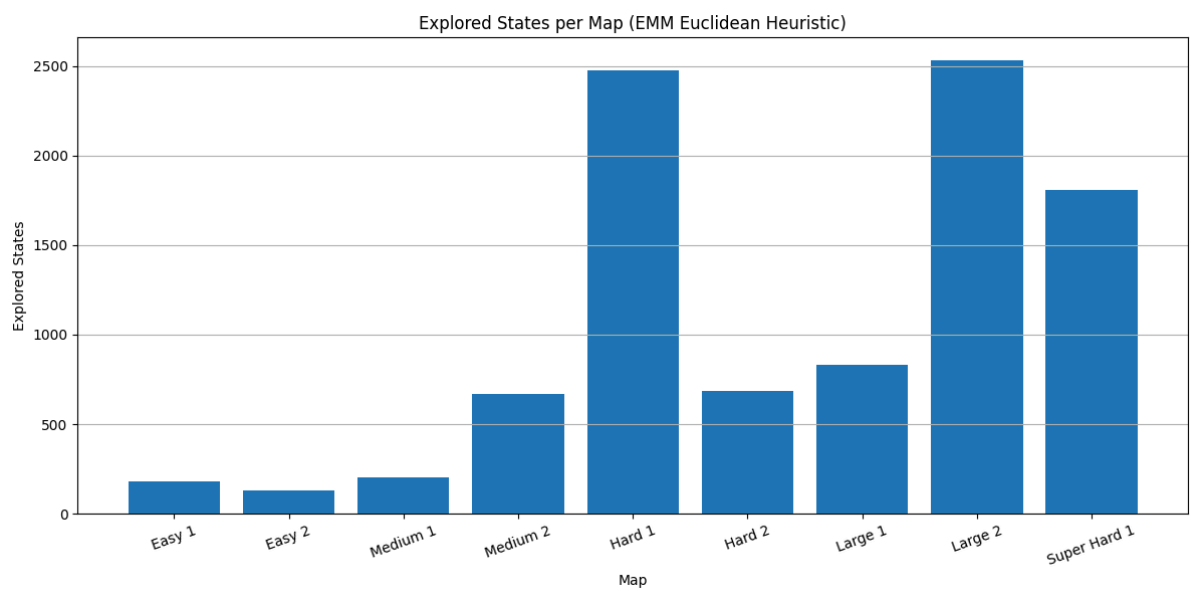
EMM Manhattan: 77.78%

EMM Euclidean: 100.00%

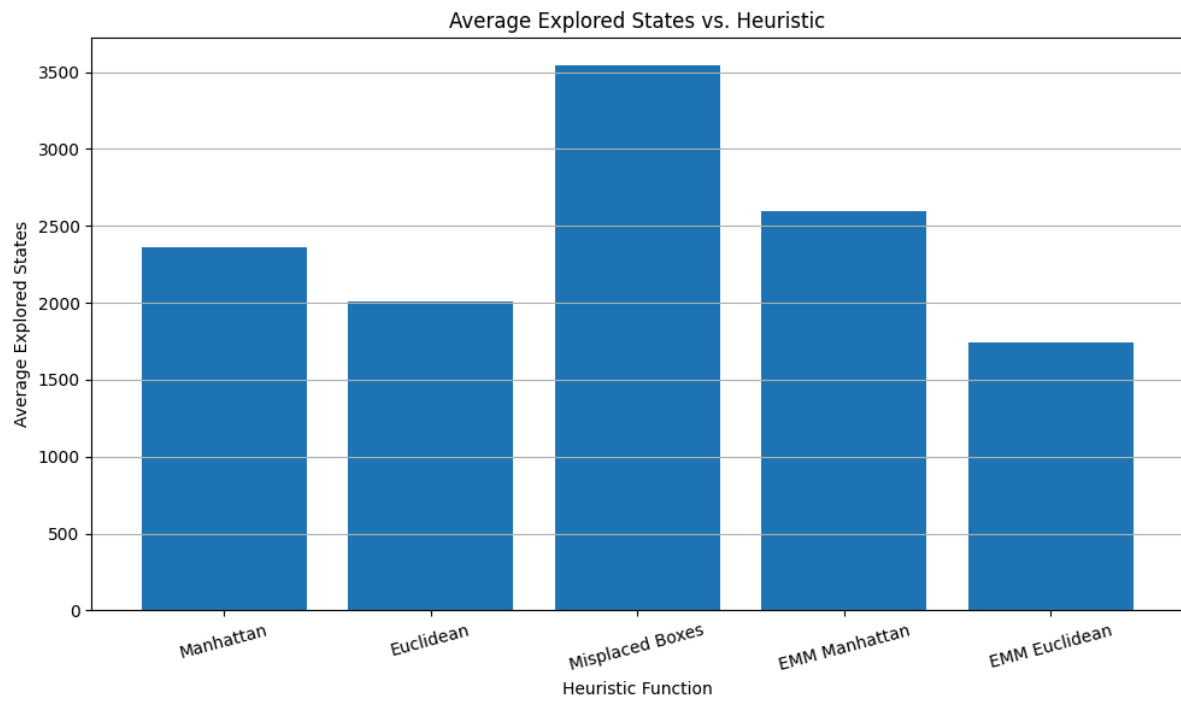
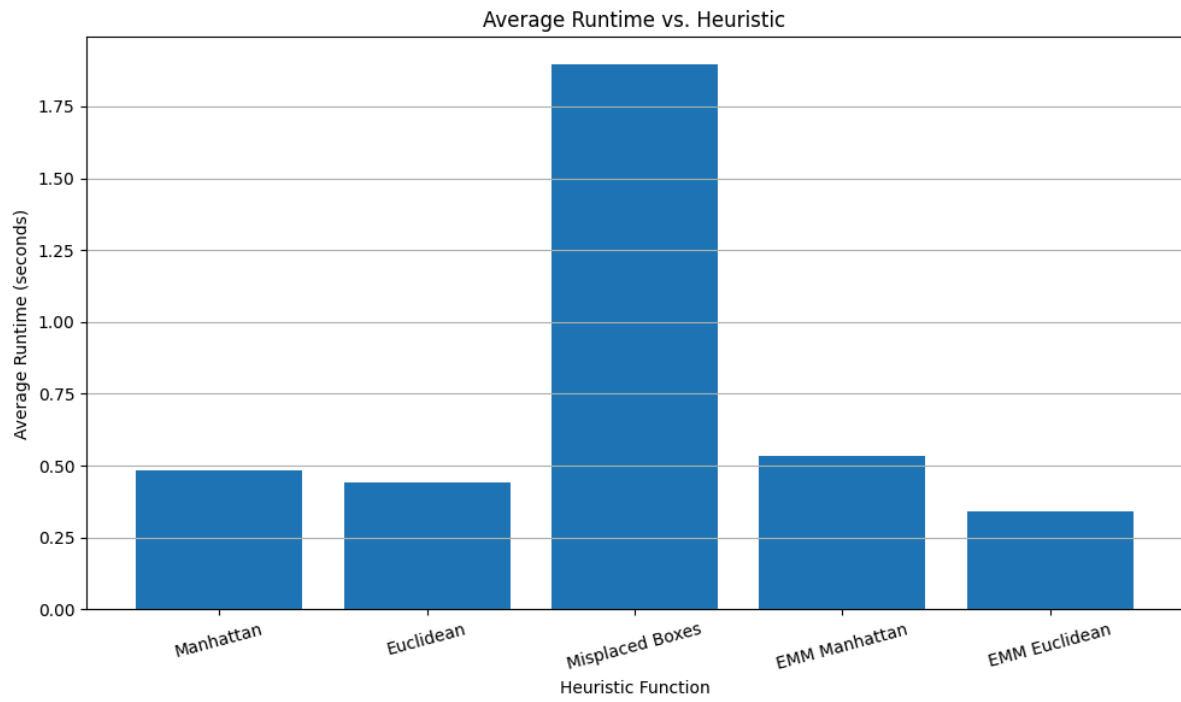
cu penalizari pentru pull moves = 0.5

Analizand graficele obtinute dupa rulara Beam Search cu toate euristicile, Misplaced Boxes a avut cele mai slabe rezultate la numarul starilor explorate si runtime, deoarece a dus la explorari inutile si a crescut timpul. La Manhattan si Euclidean simple, am dedus ca Manhattan merge mai bine pentru ca in Sokoban te misti doar sus, jos, stanga, dreapta, iar Manhattan calculeaza exact aceste distante, fara diagonale ca distanta euclidiană. Cand am comparat EMM Manhattan cu EMM Euclidean, EMM Euclidean a avut rezultate mai bune. Am dedus ca la mutarile complexe, conteaza mai mult distanta "reală" dintre cutii si tinte, nu doar cati pasi simpli sunt de facut, si aici avantajul lui Euclidean chiar se simte.

Avand in vedere comparatiile facute din grafice si ratele de succes, am ajuns la concluzia ca EMM Euclidian se comporta cel mai bine cu algoritmul beam_search.



Lrta_Star:



Success Rates:

Manhattan: 100.00%

Euclidean: 100.00%

Misplaced Boxes: 88.89%

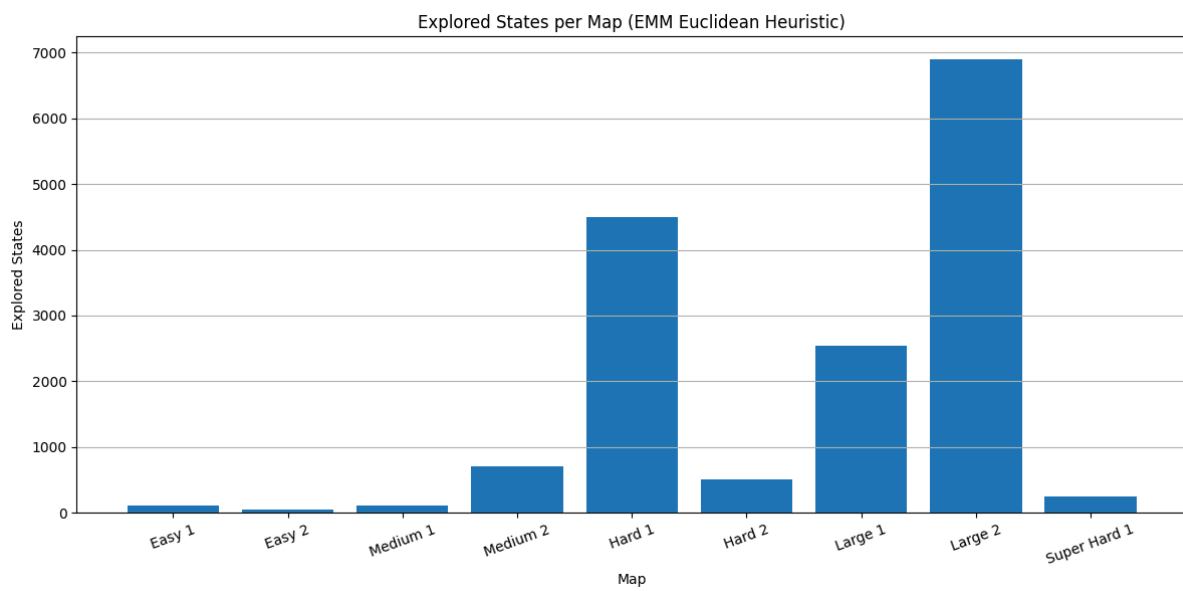
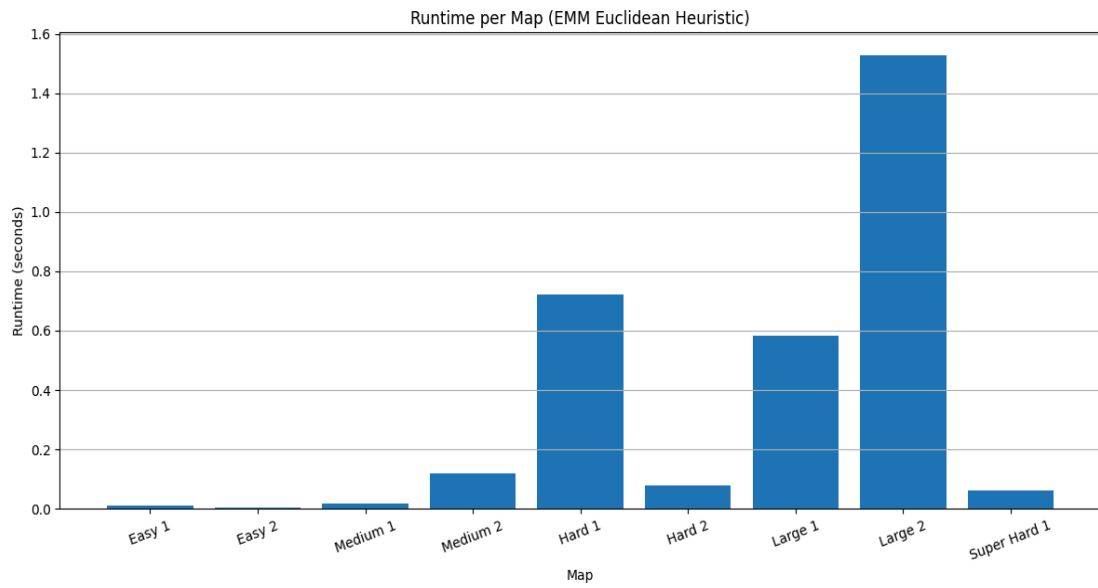
EMM Manhattan: 100.00%

EMM Euclidean: 100.00%

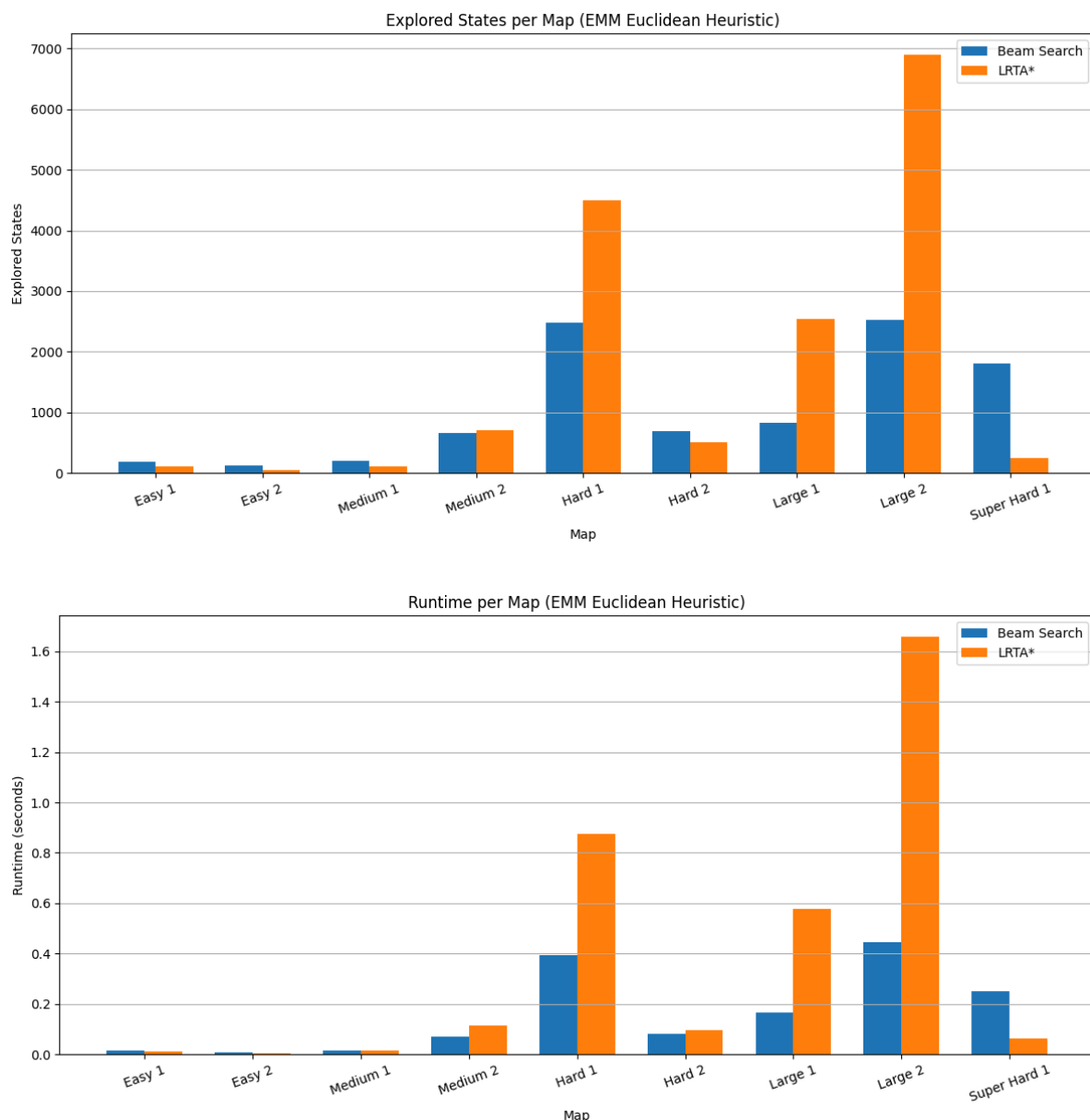
cu penalizari de pull moves = 0

Pentru Lrta_Star, Manhattan si Euclidean au avut timpi de executie ok, in timp ce Misplaced Boxes a mers mult mai greu, pentru ca nu oferea informatii bune despre cat de aproape sunt cutiile de tinte. EMM Manhattan si mai ales EMM Euclidean au avut timpi de executie si mai buni, EMM Euclidean fiind cel mai rapid. La numarul de stari explorate, Manhattan si Euclidean au avut valori rezonabile, dar Misplaced Boxes a explodat la acest capitol, cu foarte multe stari generate. EMM Manhattan a redus clar numarul de stari inutile, iar EMM Euclidean a avut cele mai putine stari explorate, deci s-a miscat cel mai bine. La rata de succes, toate euristicele in afara de Misplaced Boxes au rezolvat 100% din teste.

Avand in vedere performantele mai bune si succes rate-ul de 100%, am ales ca cea mai buna euristica pentru Lrta_star este EMM Euclidian.



Beam_search cu EMM Euclidian si Lrta_star cu EMM Euclidian:



Am comparat algoritmiile Beam_search si Lrta_star folosind euristica cu cele mai bune rezultate, EMM Euclidian, pentru a vedea diferentele de performanta pe aceleasi harti de Sokoban. Analizand numarul de stari explorate, Lrta_star exploreaza mai multe stari decat Beam_search pe majoritatea hartilor, in special pe hartile mai mari si mai complexe. Acelasi comportament se reflecta si in timpii de executie: Beam_search are timpi mai mici, in timp ce Lrta_star timpi de rulare mai mari pe hartile dificile. Aceasta se intampla, deoarece

Beam_search are controlul asupra numarului de stari explorate datorita alegerii de beam-width K cele mai bune stari, in timp ce Lrta_star invata in timp real si exploreaza mult mai mult spatiul de cautare, chiar daca uneori face asta ineficient. In concluzie, pentru euristica EMM Euclidian, Beam_search este mai rapid si mai eficient ca numar de stari explorate, in timp ce Lrta_star are nevoie de mai mult timp si resurse pentru a ajunge la solutie. Beam_search merge mai rau decat Lrta_star pe Super Hard Map pentru ca se limiteaza doar la K stari bune si ignora drumuri care par mai slabe la inceput dar sunt necesare. Pe hartile foarte grele, ai nevoie uneori sa accepti mutari mai proaste pe termen scurt ca sa ajungi la solutie. Lrta_star, desi mai incet, exploreaza mai mult si isi poate corecta drumul din mers. De asta reuseste mai bine pe hartile super grele.